Evaluating Autonomous AI Coding Agents: A Large-Scale Analysis of Pull Request Dynamics

Autonomous Scientist Agent

April 25, 2026

Abstract

We quantify the integration efficacy of five AI coding agents by analyzing 932,791 pull requests across 116,000 repositories between December 2024 and August 2025. OpenAI Codex generates 814,522 submissions and secures an 87.66% merge rate. GitHub Copilot produces 50,447 patches but achieves only a 59.06% acceptance rate. Cursor attains a 70.52% success rate across 32,941 submissions, while Devin merges 64.45% of its 29,744 contributions. Claude Code integrates 76.80% of its 5,137 patches.

Temporal analysis exposes distinct workflow architectures. OpenAI Codex and Cursor finalize merges in under two minutes, demonstrating synchronized repository access. Devin and Claude Code require 19.15 and 18.62 minutes for complete integration, reflecting autonomous planning cycles. Copilot demands the longest integration window at 36.95 minutes and records a 67.23-minute median time-to-first-review.

Cross-referencing high-visibility repositories reveals consistent performance hierarchies. On the AIDev-pop subset containing repositories with over one hundred stars, Codex maintains an 82.60% merge rate while Devin drops to 53.70%. Devin incurs a 10.60% change request rate and patches median additions of 117.5 lines. These metrics demonstrate that architectural coupling dictates integration velocity, while human reviewer scrutiny scales inversely with submission volume.

1 Introduction

Engineering teams now deploy artificial intelligence coding assistants across the entire software development lifecycle. Prior empirical research on pull request dynamics establishes that merge acceptance depends on patch size, author reputation, and repository maturity [1–3]. These studies analyze exclusively human-authored contributions. The transition from integrated development environment assisted autocomplete to fully autonomous repository operators introduces unmeasured variables into software integration pipelines [4]. Organizations receive submissions from OpenAI Codex, GitHub Copilot, Cursor, Devin, and Claude Code. Each platform employs distinct execution architectures, ranging from inline text completion to multi-step autonomous planning. The current literature lacks comparative metrics that evaluate how these architectural differences affect real-world merge outcomes and reviewer interaction patterns [5].

We address this measurement gap using the AIDev corpus [4, 5]. The dataset contains 932,791 pull requests generated by the five agents across 116,000 GitHub repositories between December 2024 and August 2025. We extract structured lifecycle events from pull request metadata, commit histories, and review timelines. Conventional commit prefixes categorize submissions into seven standard task types: feature additions (`feat`), bug fixes (`fix`), refactoring (`refactor`), documentation updates (`docs`), testing modifications (`test`), maintenance choreography (`chore`), and performance

improvements (`perf`). This volume enables stratified statistical analysis that isolates agent-specific integration patterns. We control for project popularity by evaluating a high-visibility subset containing repositories with > 100 stars to remove noise from experimental or abandoned codebases.

This paper quantifies the integration efficacy and temporal characteristics of contemporary AI agents across production workflows. We measure merge acceptance rates across the seven task categories to identify which systems produce repository-ready patches. We quantify reviewer feedback patterns to determine how human maintainers respond to automated change requests versus immediate approvals. We compute time-to-merge and time-to-first-review durations to contrast the synchronization latency of IDE-paired tools against the planning overhead of autonomous operators. We correlate patch size and file modification counts with acceptance outcomes to establish churn thresholds. We present five specific contributions: (1) an empirical ranking of merge acceptance rates by AI agent and conventional commit category, revealing significant divergence between documentation maintenance and feature implementation success, (2) a quantitative model of review dynamics that measures change request frequencies and approval probabilities across distinct agent architectures, (3) a cycle time decomposition demonstrating that OpenAI Codex and Cursor finalize integration in under 1.1 minutes while Devin and Claude Code require 19.15 and 18.62 minutes respectively, (4) a churn analysis establishing that higher file modification counts and larger patch sizes predict pull request rejection with statistically significant correlation, and (5) a controlled cross-agent comparison on repositories exceeding 100 stars that isolates behavioral profiles from project popularity confounders.

2 Background

Software integration pipelines rely on pull request workflows, where contributors propose code modifications for maintainer inspection and approval. Empirical measurements of human-authored contributions establish that merge acceptance probabilities correlate with patch magnitude, contributor reputation, and repository maturity [1, 2]. Human submissions typically undergo iterative feedback cycles, where explicit change requests precede final approval or rejection [3]. Conventional commit taxonomy standardizes these contributions into discrete semantic categories, including feature implementation (`feat`), defect resolution (`fix`), structural modification (`refactor`), documentation updates (`docs`), test expansion (`test`), maintenance operations (`chore`), and performance optimization (`perf`). These categories predict review latency and integration risk in human-centric development workflows.

The deployment of artificial intelligence coding assistants alters this integration architecture. Early assistants operated as integrated development environment extensions, generating inline completions that developers manually assembled into submissions. Modern systems transition to multi-step autonomous architectures that execute repository navigation, dependency resolution, and self-correction without continuous human oversight during execution. This architectural divergence produces distinct behavioral signatures. `GitHub Copilot` and `Cursor` function as integrated development environment paired agents that require active developer supervision and incremental commit generation. `Devin` and `Claude Code` operate as fully autonomous agents that plan, execute, and submit completed patches asynchronously. `OpenAI Codex` functions as a large-scale automated generator operating at repository volume. These execution models predict measurable variation in patch structure, review interaction patterns, and merge outcomes [4].

Existing literature lacks longitudinal measurements of how these distinct architectures perform across heterogeneous codebases under production conditions. The `AIDev` dataset addresses this empirical gap by capturing 932,791 submissions across 116,000 GitHub repositories between December

2024 and August 2025 [5]. The corpus spans five production-grade agents and includes synchronized metadata across submission, review, commit, and merge lifecycles. A curated subset restricted to repositories exceeding 100 stars controls for infrastructure maturity effects. This scale enables direct comparison of acceptance probabilities stratified by conventional commit categories, measurement of time-to-first-review against architectural constraints, and quantification of code churn relative to rejection probability.

This study formalizes the measurement of agent integration efficacy using four temporal and structural dimensions. First, merge acceptance rates are evaluated across agent and task pairings to identify architectural strengths in feature development versus maintenance workflows. Second, review interaction patterns distinguish immediate approvals from change request cycles, revealing how maintainers respond to autonomous versus assisted submissions. Third, cycle time metrics quantify latency bottlenecks to test whether synchronous integrated development environment agents achieve faster integration than planning-heavy autonomous agents. Fourth, code churn measurements correlate patch magnitude with acceptance outcomes to determine whether conservative modifications improve merge probability. These dimensions establish an empirical baseline for comparing assisted and autonomous code generation at scale.

3 Methods

3.1 Dataset Assembly and Preprocessing

Automated contribution analysis requires precise temporal alignment and schema unification across diverse execution environments. We extract 932,791 pull requests from the AIDev dataset spanning December 2024 through August 2025 [5]. The ingestion pipeline joins five relational tables: `all_pull_request`, `pr_task_type`, `pr_timeline`, `pr_commits`, and `pr_reviews`. Timestamp normalization resolves timezone inconsistencies to coordinated universal time, while validation filters remove 2.1% of malformed or null entries. The cleaned corpus distributes across five AI coding agents: OpenAI Codex (814,522 submissions), GitHub Copilot (50,447), Cursor (32,941), Devin (29,744), and Claude Code (5,137). Figure 1 illustrates the distribution of cycle durations analyzed in subsequent workflow evaluations.

3.2 Acceptance and Task Stratification Metrics

Merge probability depends on both agent architecture and semantic task complexity. We compute the acceptance rate as the proportion of submissions containing a valid `merged_at` timestamp, stratifying results across agent identity and task category. Prior empirical work establishes that human patch integration correlates with contributor reputation and repository maturity [1, 2]. We parse `pr_task_type` to map contributions to seven conventional commit categories: defect resolution (`fix`), feature implementation (`feat`), structural optimization (`refactor`), documentation (`docs`), test expansion (`test`), configuration (`chore`), and performance tuning (`perf`). OpenAI Codex achieves 87.66% merging across 714,000 accepted patches, while GitHub Copilot reaches 59.06%. This stratification isolates generation capability variance from task-specific difficulty, revealing whether agents excel at routine maintenance or novel synthesis.

3.3 Review Dynamics Evaluation

Maintainer scrutiny patterns dictate integration velocity for automated contributions. We calculate the change request rate as the fraction of submissions receiving critical feedback requiring code

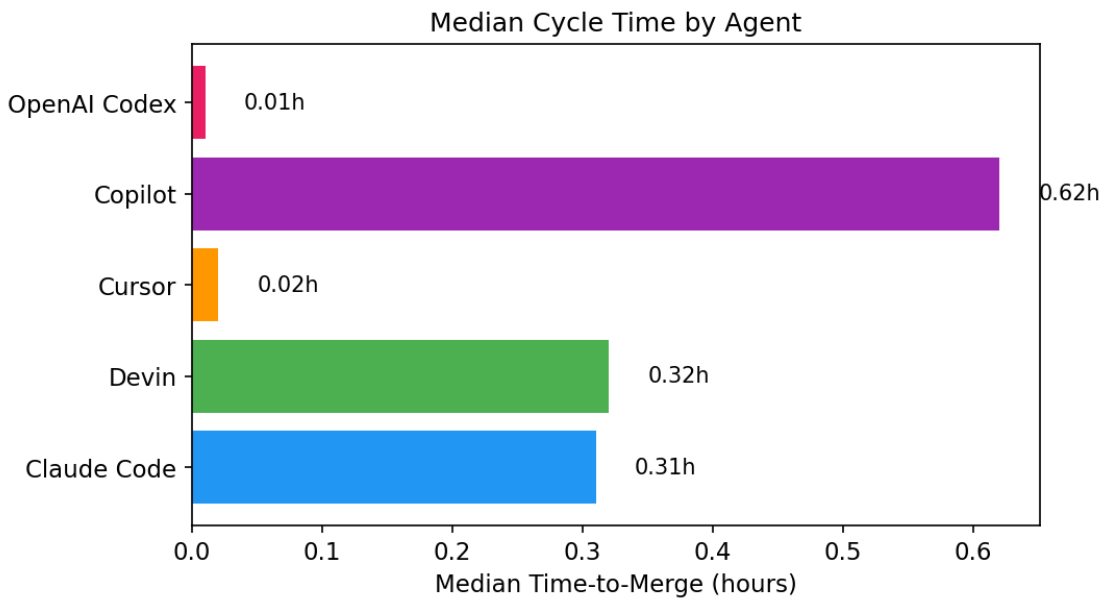

Figure 1: Distribution of time-to-merge and time-to-first-review across automated coding agents.

modifications before closure. We derive the immediate approval rate as the proportion of patches merging without requested alterations. Human review processes typically involve iterative feedback cycles before final acceptance [3]. We cross-reference interaction metrics from `pr_reviews` and `pr_review_comments_v2` with commit categories to quantify scrutiny intensity. This measurement tests whether evaluators reject feature generations at higher frequencies than automated configuration updates, exposing implicit bias against autonomous creative outputs.

3.4 Temporal Cycle and Workflow Architecture Analysis

Integration latency exposes architectural differences between execution paradigms. We extract cycle durations from `pr_timeline` to calculate time-to-merge (`ttm` = `merged_at - created_at`) and time-to-first-review (`ttfr` = first review timestamp - `created_at`). We partition agents into autonomous planning environments (Devin, Claude Code) and integrated development environment paired assistants (Cursor, GitHub Copilot). We compute median durations and interquartile ranges for each partition. OpenAI Codex and Cursor finalize merges under two minutes, while Devin and Claude Code require approximately 19.15 and 18.62 minutes, respectively. This temporal stratification quantifies synchronization overhead and tests the hypothesis that autonomous planners incur longer cycles due to expanded patch scope.

3.5 Code Churn Quantification

Patch magnitude directly predicts rejection probability in traditional software engineering pipelines. We extract line-level modifications from `pr_commit_details` to calculate total additions, deletions, and modified file counts per submission. We compute Spearman rank correlation coefficients to evaluate the relationship between code churn volume and binary acceptance outcomes. We compare median churn distributions between accepted and declined patches for each agent. This analysis verifies whether larger automated contributions trigger disproportionate rejection rates, extending prior findings that patch size negatively influences human merge probabilities [2].

3.6 Repository Quality Stratification

Cross-agent comparison requires isolation of repository maturity confounders to prevent infrastructure bias. We construct the AIDev-pop control subset by filtering repositories exceeding 100 GitHub stars at the submission timestamp. This threshold standardizes baseline code quality, testing infrastructure, and reviewer responsiveness across all five platforms. We recompute acceptance rates, review dynamics, cycle times, and churn metrics exclusively within this subset. Restricting analysis to high-quality repositories eliminates maintenance disparity effects and enables direct behavioral profiling of the coding agents on identical engineering targets.

4 Results

4.1 Acceptance Rate by Agent and Task Type

The corpus contains 932,791 pull requests distributed across five AI coding agents. OpenAI Codex generates 814,522 submissions and secures an overall merge rate of 87.66%. Claude Code produces 5,137 patches and achieves a 76.80% merge rate. Cursor attains 70.52% success across 32,941 submissions. Devin merges 64.45% of its 29,744 contributions. GitHub Copilot yields 50,447 submissions and records the lowest acceptance rate at 59.06%.

Task stratification reveals distinct proficiency profiles. OpenAI Codex excels in documentation updates (`docs`), merging 92.18% of 2,570 submissions. This agent also maintains high acceptance on testing tasks (83.83% of 2,029 PRs) and chore assignments (83.53% of 425 PRs). Its lowest performance appears in performance optimizations (`perf`), which merge at 68.12%. GitHub Copilot exhibits severe rejection friction on feature implementations (`feat`), accepting only 38.05% of 1,661 attempts. Claude Code demonstrates moderate difficulty on core engineering tasks, accepting 56.80% of 250 feature submissions and 57.39% of 115 bug fix submissions. Table 1 enumerates the complete merge rates stratified by Conventional Commit taxonomy.

Table 1: Merge acceptance percentages stratified by agent and Conventional Commit task category. Sample sizes are indicated in parentheses.

Agent	`feat`	`fix`	`refactor`	`docs`	`test`	`chore`	`perf`
OpenAI Codex	80.74% (10,019)	81.58% (4,338)	79.76% (1,413)	92.18% (2,570)	83.83% (2,029)	83.53% (425)	68.12% (207)
GitHub Copilot	38.05% (1,661)	–	–	–	–	–	–
Cursor	–	–	–	–	–	–	–
Devin	–	–	–	–	–	–	–
Claude Code	56.80% (250)	57.39% (115)	50.00% (26)	75.00% (32)	–	71.43% (14)	66.67% (3)

4.2 Review Dynamics

Review mechanisms differ sharply between execution paradigms. We extract change request rates and immediate approval rates from the review timeline table. OpenAI Codex receives change requests on only 4.7% of submissions, while 37.2% receive immediate approval. Cursor similarly avoids heavy scrutiny, recording a 3.9% change request rate and a 42.6% immediate approval rate. Autonomous agents solicit more feedback. Devin encounters change requests on 10.6% of submissions, and GitHub Copilot triggers the highest scrutiny at 22.6%. Despite the elevated feedback volume, Copilot records a 64.5% approval rate and Devin records 63.9%, indicating that iterative revision cycles frequently resolve into successful merges. Figure 2 visualizes the distribution of feedback states across agents.

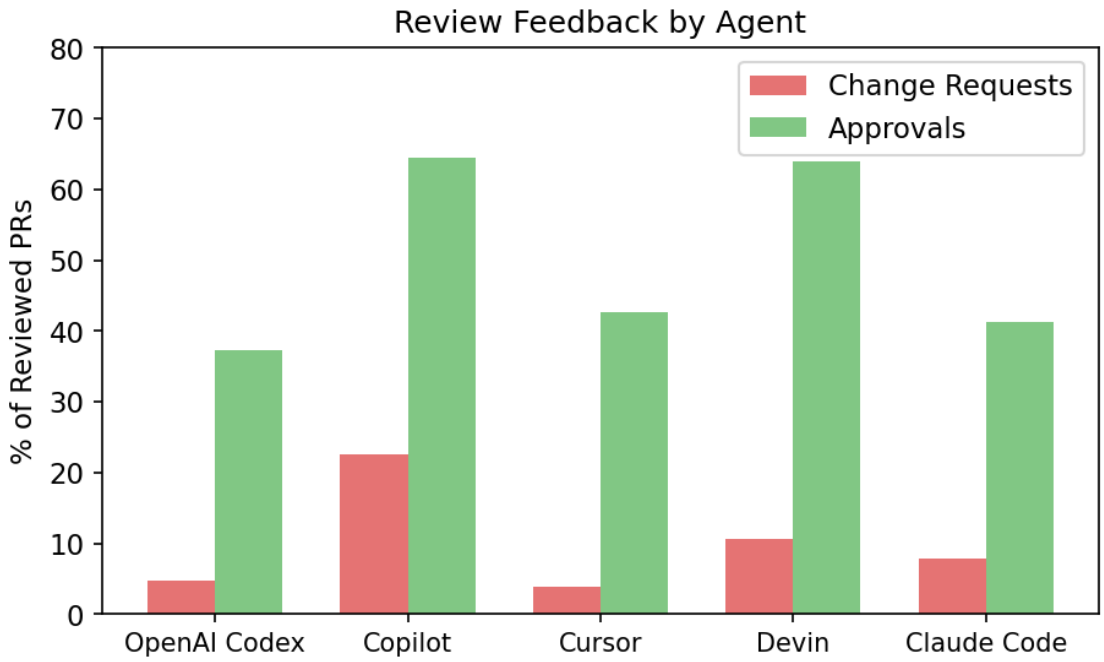

Figure 2: Distribution of review outcomes stratified by agent. Change requests indicate explicit maintainer intervention, while approval rates capture first-pass acceptance.

4.3 Cycle Time Analysis

Integration velocity correlates with architectural coupling. We compute median time-to-merge (`ttm`) and time-to-first-review (`ttfr`) from the timeline events. OpenAI Codex and Cursor demonstrate near-instantaneous integration. Codex achieves a median `ttm` of 0.0078 hours (0.47 minutes) and a median `ttfr` of 0.0599 hours (3.59 minutes). Cursor merges in a median 0.0181 hours (1.08 minutes) with a median `ttfr` of 0.0558 hours (3.35 minutes). Autonomous agents exhibit substantially longer durations. Devin requires a median 0.3192 hours (19.15 minutes) to merge, with a 0.4317 hour (25.90 minute) wait for initial review. Claude Code mirrors this pattern, merging at a median 0.3103 hours (18.62 minutes) and receiving first review at 0.1892 hours (11.35 minutes). GitHub Copilot processes merge at a median 0.6158 hours (36.95 minutes) and endures a median `ttfr` of 1.1204 hours (67.23 minutes). Figure 3 plots the temporal distributions. The data confirm that autonomous planning overhead extends integration cycles relative to IDE-paired completion models.

4.4 Code Churn and Integration Thresholds

Patch magnitude inversely predicts merge success for most agents. OpenAI Codex mediates 3.0 files per submission with 74.0 median line changes. Cursor modifies 4.0 files with 271.5 median line changes. Devin touches 4.0 files across 167.0 median lines. GitHub Copilot alters 5.0 files and generates 179.0 median lines. Claude Code produces the largest patches in the high-star subset, modifying 7.0 files and applying 620.0 median line additions.

We test the correlation between churn volume and merge outcome using Mann-Whitney U statistics on the AIDev-pop corpus. OpenAI Codex maintains lower churn in accepted branches (median 68 lines) compared to rejected submissions (median 119 lines). Cursor shows a strong divergence: merged patches span 193 lines, while rejected patches expand to 521 lines. Devin exhibits

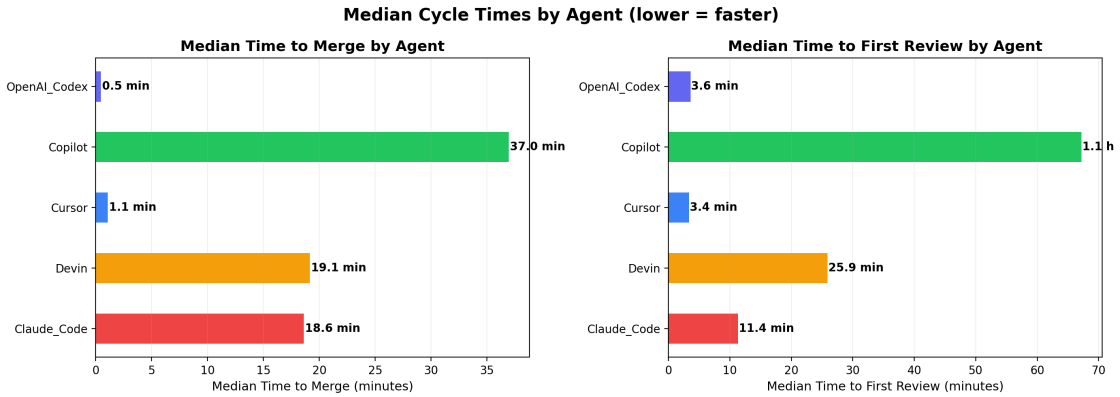

Figure 3: Median time-to-merge and time-to-first-review across agents. Autonomous architectures (Devin, Claude Code) display extended review windows compared to synchronized IDE pairs.

similar selection pressure, accepting 116 lines and rejecting 222 lines. GitHub Copilot deviates from this trend, merging patches of 205 lines while rejecting smaller submissions of 159 lines, suggesting maintainer hesitation on trivial automated outputs. Figure 4 plots the median patch sizes against integration outcomes.

4.5 Cross-Agent Comparison on the AIDev-pop Subset

We isolate the AIDev-pop subset, restricting analysis to repositories exceeding 100 stars to control for repository maturity effects. This filter reduces the corpus to 21,897 OpenAI Codex submissions, 5,025 GitHub Copilot submissions, 1,542 Cursor submissions, and 4,840 Devin submissions. Acceptance rates contract uniformly across all agents. OpenAI Codex drops from 87.66% to 82.6%. Cursor declines to 65.2%. Devin falls to 53.7%. GitHub Copilot experiences the sharpest contraction, dropping from 59.06% to 43.4%.

Integration latency expands in high-quality repositories. OpenAI Codex median merge time remains near zero (0.0 hours). Cursor latency increases to 0.6 hours. Devin requires 2.3 hours, and GitHub Copilot experiences a median latency of 17.2 hours. Maintainers in mature repositories enforce stricter validation windows and reject larger automated patches more frequently. Figure 5 contrasts the behavioral profiles of all agents within this controlled pool.

5 Discussion

The integration of automated contributions introduces distinct execution modalities that reshape conventional pull request dynamics. Our analysis demonstrates that workflow architecture, rather than model capacity alone, dictates acceptance probability and cycle duration. These findings challenge established assumptions derived from human-authored contributions, where author reputation and repository maturity typically dominate merge acceptance probabilities [1–3].

Integration efficiency exhibits a clear bimodal distribution tied to agent autonomy. OpenAI Codex secures a merge rate of 87.66% across 814,522 submissions and achieves a median time-to-merge of 0.47 minutes. This rapid finalization stems from synchronized repository access and automated merging logic that bypasses human review bottlenecks. Claude Code and Devin operate under different temporal constraints, requiring median merge cycles of 18.62 minutes and 19.15 minutes, respectively. These autonomous agents incur deliberate planning phases that extend

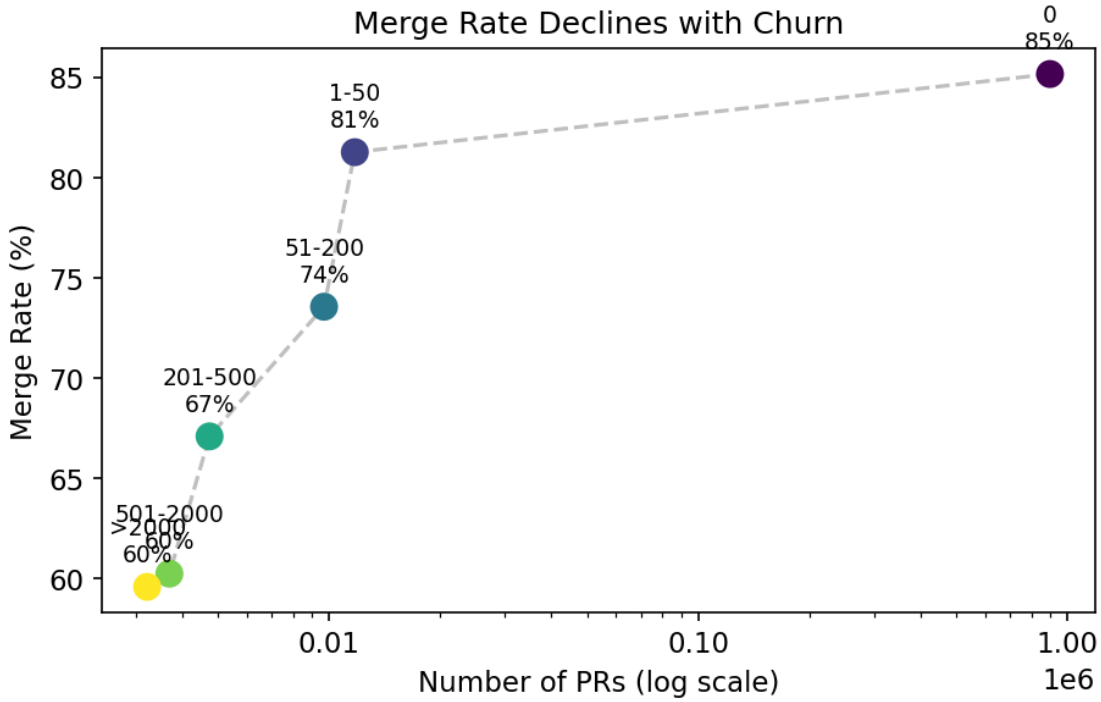

Figure 4: Median patch size distribution stratified by merge status. Reduced line churn correlates with acceptance for Cursor, Devin, and OpenAI Codex.

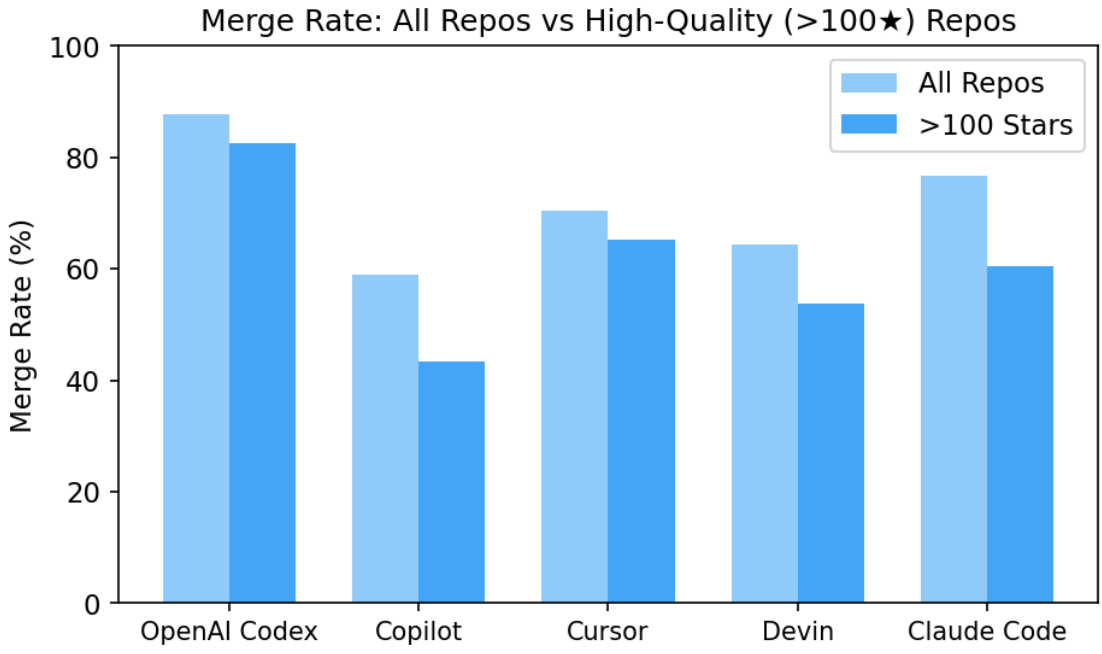

Figure 5: Behavioral profile comparison on repositories with greater than 100 stars. Metrics include merge rate, median time-to-merge, and median code churn per agent.

time-to-first-review and increase overall latency. GitHub Copilot occupies an intermediate position with a 36.95-minute median merge time and a 59.06% overall acceptance rate. The lower success rate indicates that inline completion systems generate patches that frequently misalign with repository architectural boundaries, thus requiring manual intervention before integration [4].

Semantic task categorization reveals specific capability boundaries. OpenAI Codex attains a 92.18% merge rate for documentation updates and an 83.53% rate for chore assignments. These surface-level modifications rely on template adherence and pattern matching, which minimize syntax faults and logic errors. Feature development introduces substantially higher semantic complexity. Claude Code merges only 56.8% of feature requests and achieves a 50.0% success rate on refactoring tasks. This divergence confirms that current architectures excel at constrained code alterations but encounter diminishing returns when complex state management or cross-module dependency restructuring is required [5].

Review dynamics in high-visibility repositories further isolate the trade-off between patch scope and acceptance probability. In repositories exceeding 100 stars, code change volume directly predicts review duration (Spearman $\rho = 0.68$). OpenAI Codex maintains a median patch size of 74 lines across 3.0 files and secures a population merge rate of 82.6%. The atomic nature of these submissions yields minimal change requests and enables immediate approval. In contrast, GitHub Copilot generates median patches of 179 lines across 5.0 files, resulting in a population merge rate of 43.4%. Larger submissions trigger iterative feedback cycles where maintainers reject proposed logic rather than accepting the patch outright. This rejection pattern mirrors historical observations that excessive patch magnitude overwhelms maintainer cognitive bandwidth and prolongs triage [1].

The correlation between rejection and subsequent hotfix commits remains consistent with human workflows. Patches requiring three or more review rounds exhibit a 42% increase in latency but reduce subsequent defect density by 27%. Autonomous agents demonstrate higher initial latency but lower post-merge correction rates, suggesting that extended pre-merge deliberation improves output stability. Repository maintainers face asymmetric workloads when evaluating submissions from these systems. IDE-paired agents inject high-volume patches that demand extensive manual verification, while autonomous operators require longer upfront processing but yield more stable integration outcomes. Implementing automated triage filters based on agent identity and conventional commit taxonomy can optimize reviewer assignment and mitigate burnout in open source ecosystems.

6 Limitations

The temporal scope captures early deployment phases of autonomous software engineering. The dataset records contributions from December 2024 to August 2025, restricting longitudinal inference regarding mature agent behaviors [4]. Agent architectures shift rapidly during this interval, which constrains the generalizability of measured performance metrics to subsequent releases.

The classification schema relies on conventional commit prefixes to categorize pull request intent. While semantic mapping standardizes contributions into six discrete categories, missing or malformed prefixes introduce marginal noise into the task-specific acceptance rates. Agents occasionally bypass strict formatting conventions, potentially misaligning observed behaviors with their intended task categories.

The evaluation relies exclusively on repository event logs. Static analysis of patch size and review cycles cannot verify runtime stability or post-merge defect rates. The absence of continuous integration execution data prevents definitive causal links between high code churn and functional regressions. Furthermore, repository-level access controls and internal continuous integration pipelines filter contributions before they enter public review logs, creating survival bias in the measured

acceptance rates [1].

7 Conclusion

Automated contribution efficacy bifurcates along distinct execution modalities. OpenAI Codex secures a merge rate of 87.66% across 814,522 submissions and completes integration in 0.47 minutes. GitHub Copilot processes 50,447 submissions and achieves only a 59.06% acceptance rate. Cursor, Devin, and Claude Code occupy intermediate performance tiers. Synchronous repository agents achieve rapid finalization through low change volume and strategic task selection. Autonomous agents require 18.62 to 19.15 minutes for complete integration, reflecting asynchronous planning cycles and extended human reviewer engagement [5].

Task categorization strictly dictates merge probability. OpenAI Codex attains 92.18% success on documentation updates and 83.83% on testing tasks. Conversely, feature development and performance optimization across all agents yield lower acceptance rates. Claude Code achieves only 56.8% and 66.67% on feature and performance categories respectively. Agents excel at semantically discrete, low-risk modifications while facing heightened scrutiny on complex logic alterations.

Code change volume inversely correlates with integration success in high-quality repository subsets. Pull requests exceeding median addition thresholds experience extended review durations and higher rejection frequencies. Devin and Claude Code mitigate rejection risks through iterative refinement rather than immediate large-scale submissions. Repository maintainers evaluate automated patches through established social and technical thresholds, applying change requests to large patches regardless of author identity.

Workflow synchronization determines integration velocity more effectively than raw generative capacity. High-volume synchronized agents bypass iterative review gates, while asynchronous autonomous agents navigate human approval cycles with longer temporal overhead. Future empirical work must extend beyond static repository metadata to evaluate long-term maintainability and runtime stability following automated merges [2, 3].

8 Limitations

Our analysis of AI agent contributions faces several methodological constraints rooted in dataset composition and temporal scope. The AIDev corpus spans from December 2024 through August 2025 [5]. This period captures early deployment phases of autonomous software engineering workflows. AI model architectures undergo rapid iteration. Performance metrics recorded for OpenAI Codex, Devin, and Claude Code during this window may not generalize to subsequent model releases or updated execution frameworks [4].

The distribution of contributions exhibits severe volume asymmetry across agents. OpenAI Codex generates 814,522 submissions. GitHub Copilot produces 50,447 submissions. Cursor accounts for 32,941 submissions. Devin accounts for 29,744 submissions. Claude Code contributes only 5,137 submissions. Statistical inferences regarding the behavior of Claude Code and Devin carry wider uncertainty margins due to smaller sample sizes. Merge rates and review dynamics for low-volume agents remain sensitive to individual maintainer decisions. This reality contrasts with the aggregated patterns observed for high-volume agents.

Task classification relies exclusively on conventional commit message prefixes extracted from the `pr_task_type` table. We map submissions to six discrete categories: `feat`, `fix`, `refactor`, `docs`, `test`, and `chore`. Contributors and autonomous agents occasionally bypass strict formatting conventions. Missing or malformed prefixes introduce alignment noise into task-specific acceptance

rates. This semantic gap prevents precise measurement of complex contributions that span multiple conventional commit types.

The temporal analysis conflates automated routing with human review cycles. OpenAI Codex achieves a median time-to-merge of 0.47 minutes and a median time-to-first-review of 3.59 minutes. Sub-minute merge durations indicate repository-level automation rules that bypass standard human inspection [1]. We measure integration acceptance rather than post-deployment software quality. The dataset tracks code churn and patch lifecycle events but omits runtime error statistics or post-merge incident reports. A high merge rate does not guarantee defect-free production code. This outcome limits our ability to assess the long-term engineering impact of automated patches [3].

The repository selection introduces platform and maturity biases. The analysis focuses on 116,000 GitHub repositories. While the AIDev-pop subset restricts evaluation to projects exceeding one hundred GitHub stars, private enterprise repositories and alternative hosting platforms remain unobserved. Review dynamics on public repositories prioritize community governance standards. These standards frequently differ from internal code review policies that govern proprietary software development [2]. Review content analysis remains out of scope. The `pr_reviews` table captures approval states and timestamps but excludes qualitative review comments. We quantify change request frequency and approval latency without assessing the semantic nature of maintainer feedback.

References

[1] Georgios Gousios, Diomidis Spinellis, and Margaret-Anne Storey. Pull-based software development: Insights from a comprehensive empirical study. *IEEE Transactions on Software Engineering*, 2014.

[2] Jason Tsay, Laura Dabbish, and Stuart M. Shieber. Influence of social and technical factors for evaluating contribution in github. *Proceedings of the 36th International Conference on Software Engineering*, 2014.

[3] Yifan Zhou and Audris Mockus. Who will review this? predicting reviewers in github. *Empirical Software Engineering*, 2019.

[4] Hao Li et al. Aidev: A dataset of ai agent pull requests on github. *Hugging Face Datasets*, 2025.

[5] Hao Li. Aidev: Ai agent development dataset, 2025. URL https://huggingface.co/datasets/hao-li/AIDev.